

DADA Cloud Console

ADR + PRD for v1 + Roadmap v1-v4

Part 1. ADR: GitOps-backed self-service console without PR approval

ADR-001: DADA Cloud Console architecture

Status

Proposed for implementation.

Context

DADA platform already has production-grade platform abstractions, including `ServiceDatabase`. The long-term goal is to build a Google Cloud-like console where a developer or client can create and manage platform resources through a UI:

- applications;
- PostgreSQL databases;
- gateway endpoints;
- domains;
- backups;
- restores;
- service access;
- later Redis, RabbitMQ, object storage, AI workers and other managed resources.

The system must feel like a cloud console. A user should not have to open a pull request manually or wait for platform engineers to approve routine actions. At the same time, the platform must remain stable, auditable, GitOps-compatible and safe.

A pure UI → Kubernetes API model is rejected because it bypasses Git as the source of truth and makes drift/audit/rollback harder.

A pure UI → PR → approval → merge model is also rejected for regular client actions because it does not feel like a cloud product.

The desired model is:

```
UI → Backend → Database → Worker Job → Git desired state → Argo CD →  
Kubernetes CRD → Controllers → Status back to UI
```

Decision

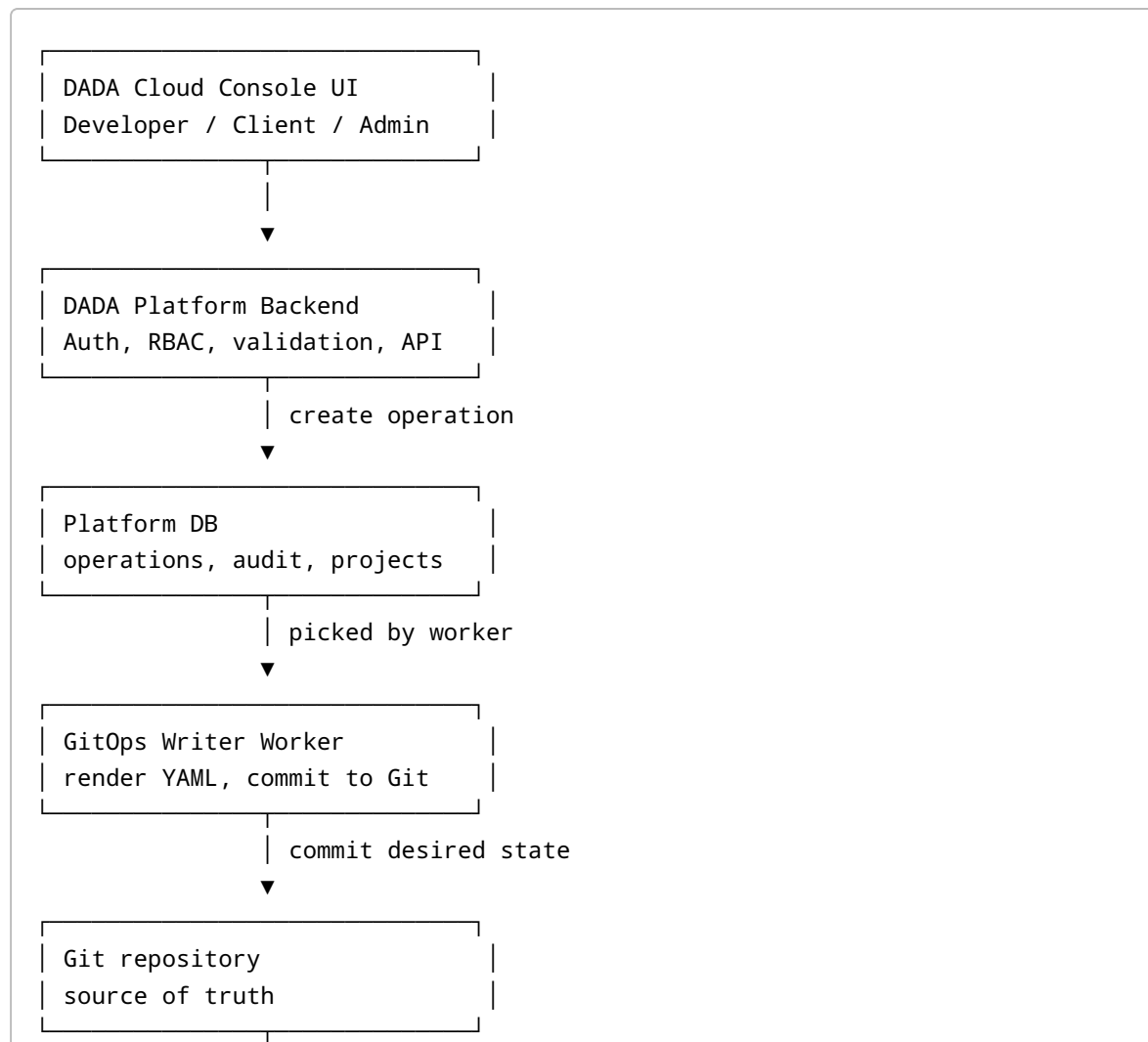
DADA Cloud Console will use a GitOps-backed asynchronous operation model.

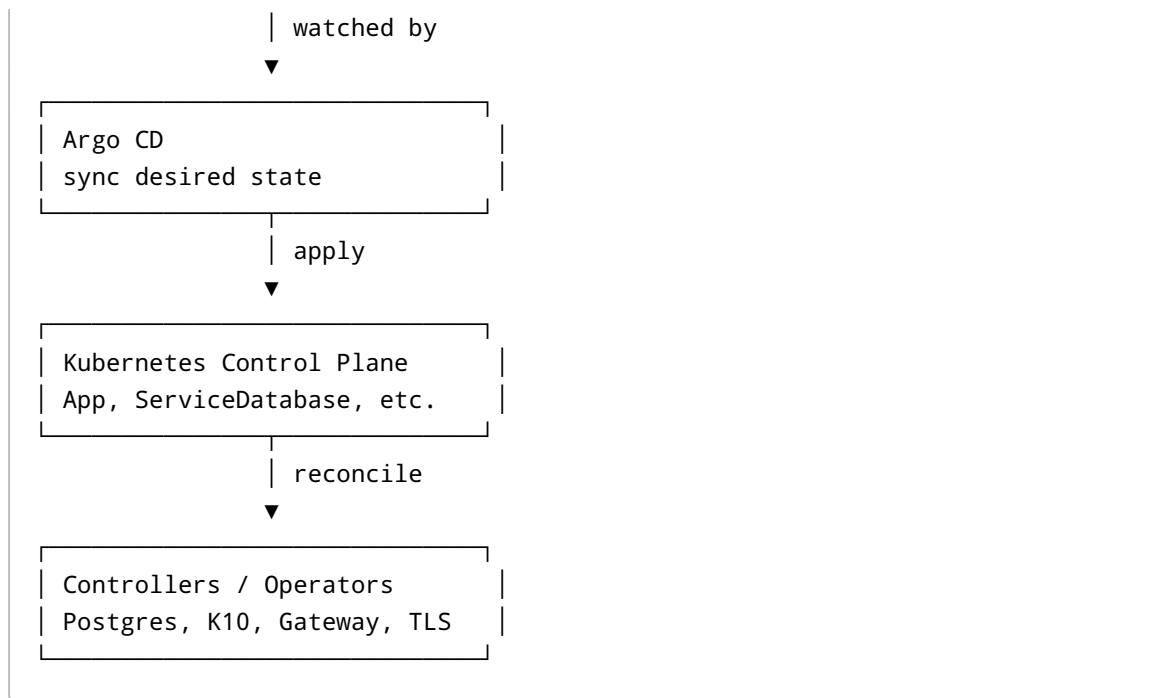
The UI will not write directly to Kubernetes and will not require the user to manually approve pull requests for standard actions.

Instead:

1. The user performs an action in the UI.
2. The backend validates the request.
3. The backend creates an **Operation** record in its database.
4. A worker picks up the operation.
5. The worker renders platform CR manifests.
6. The worker commits generated desired state into the GitOps repository.
7. Argo CD synchronizes the desired state into Kubernetes.
8. Platform controllers reconcile real infrastructure.
9. The backend aggregates live status from Kubernetes, Argo CD and platform CRDs.
10. The UI shows progress and final status.

Target flow





Consequences

Positive

- Users get a cloud-like self-service experience.
- Git remains the source of truth.
- Argo CD remains the delivery engine.
- All changes are auditable through operation records and Git commits.
- UI does not need cluster-admin access.
- The platform can recover from temporary failures because operations are asynchronous and retryable.
- Dangerous actions can still require stronger confirmation or admin approval internally, without exposing PR mechanics to the client.

Negative

- More moving parts than a simple UI → Kubernetes API model.
- Requires operation state machine.
- Requires idempotent Git writer.
- Requires conflict handling for concurrent operations.
- Requires status aggregation.

Non-goals

For v1, DADA Cloud Console will not be:

- a general Kubernetes dashboard;
- a replacement for Argo CD;
- a replacement for Grafana;
- a raw YAML editor for clients;
- a cluster-admin interface;
- a billing platform;

- a full public cloud competitor.
-

ADR-002: Git remains source of truth, but PR approval is not part of normal client flow

Status

Proposed.

Context

Classic GitOps usually implies human review through pull requests. This is useful for infrastructure teams, but it is a poor fit for a cloud console UX.

A client expects to click “Create database” and receive a database. They do not expect to approve a pull request.

However, the platform still needs:

- auditability;
- rollback;
- desired state tracking;
- drift correction;
- reproducible infrastructure;
- Argo CD integration.

Decision

Standard user actions will result in direct automated commits made by the DADA Platform Bot.

The user will not interact with pull requests directly.

Each commit must include metadata:

```
actor: user/client who requested the action
operationId: internal operation identifier
project: target project
resource: target resource
reason: action requested through DADA Cloud Console
```

Commit message example:

```
[DADA Console] Create ServiceDatabase codex-lb-db for project internal

Operation: op_01HX...
Actor: alex
Project: internal
```

Environment: prod
Resource: ServiceDatabase/prod/codex-lb-db

Exceptional approval flow

Some dangerous actions may require internal approval, but this approval should happen inside DADA Cloud Console, not through a raw Git PR exposed to the client.

Examples:

- delete production database;
- restore over existing production database;
- disable backups for production;
- expose internal service publicly;
- change auth mode from protected to public;
- rotate critical credentials;
- delete production application.

For these actions:

User request → Operation status: WaitingForApproval → Admin approves in Console → Worker commits to Git

Consequences

- Git remains authoritative.
- User experience remains cloud-like.
- Dangerous changes are still controlled.
- The platform can introduce PRs later for internal developer workflows if needed, but not as the default client flow.

ADR-003: Operation-first backend model

Status

Proposed.

Context

Creating cloud resources is not instant. Even if the UI action is quick, the actual process includes:

- validation;
- Git commit;
- Argo CD sync;
- Kubernetes apply;
- controller reconciliation;
- external resource creation;
- status propagation.

The backend should not block a request until everything is ready.

Decision

Every mutating user action creates an `Operation`.

Example operation states:

```
Created
Validated
Queued
Rendering
CommittingToGit
Committed
WaitingForArgoSync
Syncing
Reconciling
Ready
Failed
Cancelled
WaitingForApproval
```

The UI tracks the operation and shows progress.

Operation example

```
{
  "id": "op_01HXABC",
  "actorId": "user_123",
  "projectId": "internal",
  "environment": "prod",
  "action": "CreateServiceDatabase",
  "resourceKind": "ServiceDatabase",
  "resourceName": "codex-lb-db",
  "status": "Reconciling",
  "gitCommit": "abc123",
  "argoApplication": "internal-prod",
  "createdAt": "2026-05-06T15:00:00Z",
  "updatedAt": "2026-05-06T15:01:20Z"
}
```

Consequences

- UI can display progress reliably.
- Failed steps can be retried.
- Platform can handle asynchronous provisioning.
- Audit trail becomes first-class.

ADR-004: Typed product actions instead of raw YAML

Status

Proposed.

Context

Allowing users or clients to submit arbitrary YAML is dangerous.

It can lead to:

- privilege escalation;
- resource abuse;
- broken Git state;
- invalid manifests;
- accidental access to system namespaces;
- secrets exposure;
- bypassing platform policies.

Decision

DADA Cloud Console will expose typed product-level actions.

Examples:

```
CreateApplication
CreateServiceDatabase
CreateServiceEndpoint
EnableBackup
CreateRestoreRequest
ScaleApplication
DeployImageVersion
RotateSecret
```

The backend translates these typed actions into platform CRDs.

Raw YAML can exist only in admin/debug mode and only for trusted platform administrators.

Consequences

- Better security.
- Better UX.
- Stronger validation.
- Stable platform API.
- Easier client-facing productization.

ADR-005: One backend, multiple views

Status

Proposed.

Context

DADA Cloud Console must serve three audiences:

1. Clients.
2. Developers.
3. Platform administrators.

They need different levels of detail, but they should operate on the same underlying model.

Decision

There will be one Platform Backend and one product model, but role-based UI views.

Client view

Shows:

- applications;
- databases;
- domains;
- backups;
- restores;
- usage/limits;
- access keys;
- simple status.

Hides:

- Kubernetes;
- Argo CD internals;
- K10 internals;
- controller details;
- YAML;
- namespaces unless exposed as “environment”.

Developer view

Shows:

- services;
- image versions;
- deploy status;
- logs links;
- OpenAPI links;

- database status;
- gateway status;
- backup status;
- Argo sync summary;
- Grafana links.

Admin view

Shows:

- generated manifests;
- Git paths;
- Argo application names;
- child resources;
- Kubernetes events;
- conditions;
- reconcile errors;
- K10 resources;
- advanced operations.

Consequences

- One platform can serve both internal and external users.
- Clients receive a clean cloud console.
- Developers receive enough operational context.
- Admins can debug without leaving the platform.

Part 2. PRD: DADA Cloud Console v1

Product name

DADA Cloud Console.

One-line description

A GitOps-backed self-service cloud console for creating and managing DADA platform resources such as applications, PostgreSQL databases, domains and backups.

Product vision

DADA Cloud Console should make the DADA platform feel like a private cloud.

A user should be able to create an application, attach a database, expose it through a domain and enable backups without writing Kubernetes YAML, Helm values, K10 policies or Argo CD manifests manually.

The platform should stay GitOps-native, secure, auditable and stable.

Target users

Primary user 1: Internal developer

Needs to:

- create application resources;
- attach databases;
- expose services;
- inspect deploy status;
- view links to Swagger, Grafana, logs and Argo;
- perform routine operational actions without asking platform engineer every time.

Primary user 2: Client

Needs to:

- see their applications;
- create or request managed resources;
- manage domains;
- see database and backup status;
- trigger safe restore workflows;
- understand service health without knowing Kubernetes.

Primary user 3: Platform admin

Needs to:

- debug failed operations;
- inspect generated manifests;
- inspect Git commits;
- inspect Argo/Kubernetes/controller status;
- approve dangerous operations;
- enforce platform policies.

Problem statement

Current platform operations require too much low-level infrastructure knowledge.

To create a fully usable service, a user may need to understand:

- Kubernetes resources;
- Helm charts;
- Argo CD applications;
- GitOps repository layout;
- PostgreSQL provisioning;
- K10 backup resources;
- gateway routing;
- TLS/cert-manager;
- service status and observability.

This is acceptable for a platform engineer, but not for a client or regular developer.

Goals for v1

DADA Cloud Console v1 must prove the core self-service loop:

User clicks → backend creates operation → worker commits desired state to Git
→ Argo applies → platform reconciles → UI shows status

v1 should support:

1. authentication;
2. projects;
3. environments;
4. applications as first-class resources;
5. PostgreSQL database creation through existing `ServiceDatabase` ;
6. basic gateway/domain creation;
7. backup enablement/status;
8. operation tracking;
9. Git-backed desired state;
10. role-based views for client, developer and admin.

Non-goals for v1

v1 will not include:

- full billing;
- complex marketplace;
- Kubernetes raw object management;
- arbitrary YAML apply;
- multi-cloud support;
- advanced cost analytics;
- advanced autoscaling management;
- complete IAM service;
- full backup restore automation for all cases;
- public SaaS-grade tenant isolation guarantees.

Core user stories

Client user stories

Story C1: View my applications

As a client, I want to see all applications that belong to my project so that I understand what is running.

Acceptance criteria:

- user sees only applications from their projects;
- each app has status: Running, Pending, Failed or Degraded;

- each app shows environment, domain and basic health;
- user does not see Kubernetes internals.

Story C2: Create application

As a client, I want to create a new application from a guided form.

Acceptance criteria:

- user enters name, environment, image/source template and size profile;
- backend validates name, quota and permissions;
- operation is created;
- worker commits generated `App` manifest to Git;
- UI shows progress;
- final status becomes Ready or Failed.

Story C3: Add PostgreSQL database

As a client, I want to add a PostgreSQL database to my app.

Acceptance criteria:

- user selects an app;
- user chooses database name and backup option;
- backend creates operation;
- worker commits `ServiceDatabase` manifest;
- UI shows database status;
- connection secret is not shown in plain text by default.

Story C4: Add domain

As a client, I want to expose my app through a domain.

Acceptance criteria:

- user enters domain;
- backend validates ownership/allowed domain pattern;
- worker commits `ServiceEndpoint` manifest;
- UI shows TLS and route status;
- endpoint becomes clickable when ready.

Story C5: View backup status

As a client, I want to know whether backups are enabled and successful.

Acceptance criteria:

- user sees backup enabled/disabled;
- user sees last successful backup time if available;
- user sees restore availability;
- user does not see K10 internals.

Developer user stories

Story D1: View technical service status

As a developer, I want to see app, database, gateway, deploy and backup status in one place.

Acceptance criteria:

- service page shows app status;
- image version is visible;
- database status is visible;
- endpoint status is visible;
- Argo sync summary is visible;
- links to Grafana/logs/Swagger are available when configured.

Story D2: Deploy new image version

As a developer, I want to update the image version of an app.

Acceptance criteria:

- user enters/selects new image tag;
- backend validates allowed registry and tag format;
- worker commits updated `App` manifest;
- Argo deploys;
- UI tracks rollout status.

Admin user stories

Story A1: Inspect failed operation

As an admin, I want to inspect failed operations.

Acceptance criteria:

- admin sees operation steps;
- admin sees validation error or Git/Argo/Kubernetes error;
- admin sees related commit;
- admin sees related platform resource;
- admin can retry safe failed operations.

Story A2: Approve dangerous operation

As an admin, I want to approve dangerous changes inside the console.

Acceptance criteria:

- dangerous operation enters WaitingForApproval;
- admin sees diff/summary;
- admin can approve or reject;
- only after approval worker commits to Git.

Functional requirements

FR-001 Authentication

The console must support login through OIDC/Keycloak.

FR-002 Role-based access

The console must support at least:

```
client-viewer
client-admin
developer
platform-admin
```

FR-003 Projects

The console must support projects as ownership and tenancy boundaries.

FR-004 Environments

The console must support environments, at minimum:

```
dev
prod
```

FR-005 Applications

The console must allow creating and viewing application resources.

FR-006 ServiceDatabase

The console must support creating and viewing PostgreSQL databases through `ServiceDatabase`.

FR-007 ServiceEndpoint

The console must support creating and viewing public/internal service endpoints.

FR-008 Backups

The console must support enabling backup configuration at creation time and displaying backup status.

FR-009 Operations

Every mutating action must create an operation record.

FR-010 Git writer

The worker must render deterministic manifests and commit them to Git.

FR-011 Status aggregation

The backend must aggregate status from:

- operation DB;
- Git commit state;
- Argo CD sync status;
- platform CRD `.status`;
- Kubernetes events where needed.

FR-012 Audit log

The console must keep an audit log for all mutating actions.

Non-functional requirements

NFR-001 Stability

Operations must be idempotent and retryable.

NFR-002 Security

The browser must never talk directly to Kubernetes API.

NFR-003 Least privilege

Backend Kubernetes access must be read-oriented and limited to required resources.

NFR-004 GitOps compatibility

Desired state must be persisted in Git.

NFR-005 Observability

Backend and worker must expose metrics and structured logs.

NFR-006 Recoverability

If the worker crashes mid-operation, another worker must be able to resume safely.

NFR-007 Concurrency control

Concurrent operations against the same resource must be serialized or conflict-detected.

NFR-008 No raw YAML for clients

Clients must not be able to submit arbitrary Kubernetes YAML.

v1 screens

Screen 1: Login

OIDC login.

Screen 2: Projects

List of projects available to the current user.

Screen 3: Project overview

Shows:

- apps;
- databases;
- domains;
- recent operations;
- basic health.

Screen 4: Application list

Shows:

- name;
- environment;
- status;
- domain;
- database attached or not;
- last operation.

Screen 5: Application details

Shows:

- status;
- image;
- environment;
- endpoints;
- databases;
- backups;
- recent operations;
- developer/admin links depending on role.

Screen 6: Create application wizard

Steps:

1. Basic info.
2. Runtime/image.
3. Size/profile.
4. Add-ons: database, endpoint, backup.
5. Review.
6. Create.

Screen 7: Database details

Shows:

- database name;
- owner/user;
- status;
- connection secret reference;
- backup status;
- restore availability.

Screen 8: Domain/endpoint details

Shows:

- domain;
- route status;
- TLS status;
- auth mode;
- OpenAPI status.

Screen 9: Operations

Shows operation timeline:

Created → Validated → Queued → Committed → Argo Syncing → Reconciling → Ready

Screen 10: Admin debug view

Only platform admins.

Shows:

- generated manifests;
- Git path;
- commit hash;
- Argo app;
- Kubernetes resource names;
- conditions;

- events;
- errors.

Data model

Project

```
id
name
displayName
ownerType
ownerId
defaultEnvironment
quotas
createdAt
updatedAt
```

Environment

```
id
projectId
name
namespace
type: dev | prod
createdAt
updatedAt
```

Operation

```
id
actorId
projectId
environmentId
action
resourceKind
resourceName
status
payload
validationResult
gitCommit
gitPath
argoApplication
errorCode
errorMessage
createdAt
updatedAt
```

AuditEvent

```
id
actorId
projectId
operationId
action
resourceKind
resourceName
metadata
createdAt
```

ResourceSnapshot

Optional cache for UI speed.

```
id
projectId
environmentId
kind
name
phase
summaryJson
lastSyncedAt
```

API sketch

Projects

```
GET /api/projects
GET /api/projects/{projectId}
```

Applications

```
GET /api/projects/{projectId}/apps
POST /api/projects/{projectId}/apps
GET /api/projects/{projectId}/apps/{appName}
PATCH /api/projects/{projectId}/apps/{appName}/image
```

Databases

```
GET /api/projects/{projectId}/databases
POST /api/projects/{projectId}/apps/{appName}/databases
GET /api/projects/{projectId}/databases/{databaseName}
```

Endpoints

```
GET /api/projects/{projectId}/endpoints
POST /api/projects/{projectId}/apps/{appName}/endpoints
GET /api/projects/{projectId}/endpoints/{endpointName}
```

Operations

```
GET /api/projects/{projectId}/operations
GET /api/operations/{operationId}
POST /api/operations/{operationId}/retry
POST /api/operations/{operationId}/approve
POST /api/operations/{operationId}/reject
```

Git layout for v1

Recommended desired state repo:

```
dada-platform-state/
clusters/
  beget-prod/
    projects/
      internal/
        project.yaml
        environments/
          dev/
            apps/
              codex-lb/
                app.yaml
                database.yaml
                endpoint.yaml
            prod/
              apps/
                codex-lb/
                  app.yaml
                  database.yaml
                  endpoint.yaml
      client-a/
        project.yaml
        environments/
          prod/
            apps/
              restaurant-bot/
                app.yaml
                database.yaml
                endpoint.yaml
```

Generated resources for v1

App

```
apiVersion: platform.dada-tuda.ru/v1alpha1
kind: App
metadata:
  name: codex-lb
  namespace: internal-prod
spec:
  project: internal
  image: ghcr.io/dada-tuda/codex-lb:1.14.2
  port: 8080
  replicas: 2
  profile: small
```

ServiceDatabase

```
apiVersion: platform.dada-tuda.ru/v1alpha1
kind: ServiceDatabase
metadata:
  name: codex-lb-db
  namespace: internal-prod
spec:
  appRef:
    name: codex-lb
  engine: postgres
  database: codexlb
  backup:
    enabled: true
    schedule: daily
    retention: 14d
```

ServiceEndpoint

```
apiVersion: platform.dada-tuda.ru/v1alpha1
kind: ServiceEndpoint
metadata:
  name: codex-lb-public
  namespace: internal-prod
spec:
  appRef:
    name: codex-lb
  domain: codex-lb.dada-tuda.ru
  tls:
    enabled: true
  auth:
    mode: jwt
```

```
openapi:
  enabled: true
```

Security requirements for v1

Access model

The backend must enforce access at project level.

A user can only see and mutate resources belonging to projects assigned to them.

Service accounts

Use separate credentials for:

- Git writer;
- Kubernetes status reader;
- Argo status reader;
- admin/debug operations if needed.

Prohibited in v1

- direct browser access to Kubernetes;
- direct client access to Argo CD;
- arbitrary YAML apply;
- exposing raw secrets;
- cluster-admin token in backend;
- wildcard resource creation;
- arbitrary namespace selection;
- arbitrary domain selection without validation.

Dangerous actions

The following actions must not be one-click in v1:

- delete production database;
- restore over production database;
- disable production backups;
- make internal service public;
- delete production app.

They can exist as disabled buttons, admin-only flows, or future roadmap items.

Success metrics for v1

Product metrics

- User can create an application with database and endpoint without writing YAML.
- User can see operation progress.
- User can understand failure reason without reading Kubernetes events.

- Platform admin can trace any action to Git commit and actor.

Technical metrics

- 100% of mutating actions create audit events.
- 100% of desired state changes are committed to Git.
- Worker operations are retryable.
- No client-facing endpoint has direct Kubernetes credentials.
- Failed operations include actionable error messages.

v1 implementation phases

Phase 1: Read-only console

- Auth.
- Projects.
- App list.
- ServiceDatabase list.
- Status aggregation.
- Admin debug view.

Phase 2: Operation engine

- Operation DB schema.
- Operation state machine.
- Worker skeleton.
- Audit log.

Phase 3: Git writer

- Deterministic YAML renderer.
- Git commit integration.
- Git path conventions.
- Conflict detection.

Phase 4: Create ServiceDatabase

- UI form.
- Backend validation.
- Worker generates `ServiceDatabase`.
- Argo sync tracking.
- Status tracking.

Phase 5: Create App + Database + Endpoint

- Create application wizard.
 - Add-on selection.
 - Generated manifests.
 - Operation progress.
-

Part 3. Roadmap v1-v4

Version 1: Foundation release

Theme

Prove the core cloud-console loop with GitOps-backed operations.

Main promise

A user can create and view basic platform resources through the UI, while Git remains the source of truth.

Features

v1.1 Authentication and projects

- OIDC/Keycloak login.
- Project model.
- Environment model.
- Basic role mapping.
- Project-scoped resource visibility.

v1.2 Read-only status console

- Application list.
- ServiceDatabase list.
- Endpoint list.
- Basic status aggregation.
- Links to Argo/Grafana/Swagger for developers/admins.

v1.3 Operation engine

- Operation table.
- Operation state machine.
- Audit log.
- Worker queue.
- Retry model.

v1.4 Git writer

- Deterministic YAML generation.
- Bot commits to Git.
- Commit metadata.
- Conflict detection.
- Idempotency keys.

v1.5 Create ServiceDatabase

- Create database form.
- Attach to app/project.

- Backup toggle.
- Git commit.
- Argo sync tracking.
- Ready/Failed status.

v1.6 Create App + Database + Endpoint wizard

- Application creation.
- Optional PostgreSQL.
- Optional public endpoint.
- Optional backup.
- Review screen.
- Operation timeline.

Exit criteria for v1

- A non-admin user can create an app with PostgreSQL and endpoint without writing YAML.
 - Desired state lands in Git.
 - Argo applies it.
 - UI shows status.
 - Admin can trace operation to actor and commit.
-

Version 2: Managed service expansion

Theme

Turn DADA Cloud Console from a database/app UI into a real managed service console.

Main promise

Users can add common platform resources to applications as managed add-ons.

Features

v2.1 Better application lifecycle

- Update image tag.
- Scale replicas.
- Restart rollout.
- View deployment history.
- Roll back to previous image version.

v2.2 Advanced gateway/domain management

- Multiple domains per app.
- Internal/public endpoint modes.
- Auth mode selection.
- OpenAPI registration.
- TLS status.
- Domain validation workflow.

v2.3 Restore requests

- View restore points.
- Create restore request.
- Restore into new database.
- Admin approval for restore-over-existing.
- Restore operation timeline.

v2.4 Redis managed service

- Add Redis to app.
- Size profiles.
- Connection secret.
- Status.

v2.5 RabbitMQ managed service

- Add queue/vhost/user.
- Basic permissions.
- Connection secret.
- Status.

v2.6 Secret rotation

- Rotate app database password.
- Rotate endpoint/service tokens.
- Audit secret rotation.
- Masked secret display.

Exit criteria for v2

- User can manage app lifecycle from UI.
- User can manage domains and endpoints.
- User can create restore requests.
- Redis and RabbitMQ are available as add-ons.

Version 3: Multi-tenant client platform

Theme

Make the platform safe and usable for real external clients.

Main promise

Clients can manage their own resources with strong isolation, quotas and a polished product experience.

Features

v3.1 Tenant model

- Tenant entity.
- Tenant/project relationship.
- Client admin role.
- Client viewer role.
- Tenant-scoped audit log.

v3.2 Quotas and limits

- App count limits.
- Database count limits.
- Storage limits.
- Endpoint limits.
- CPU/RAM profile restrictions.
- Quota exceeded errors.

v3.3 Plans

- Free/internal/small/medium/custom plans.
- Plan-based resource profiles.
- Plan-based backup retention.
- Plan-based endpoint limits.

v3.4 Client-facing UX polish

- Product-style dashboard.
- Simplified health indicators.
- Usage cards.
- Onboarding wizard.
- Better empty states.
- Human-readable errors.

v3.5 Notifications

- Email/Telegram/Webhook notifications.
- Operation failed.
- Backup failed.
- Restore completed.
- Domain/TLS issue.

v3.6 Approval center

- Internal console approval flow.
- Dangerous operation approvals.
- Approval audit.
- No exposed PR mechanics.

Exit criteria for v3

- External client can safely use the console.

- Tenant isolation exists at backend and resource model level.
 - Quotas prevent resource abuse.
 - Dangerous operations are controlled through console-native approval.
-

Version 4: Full private cloud experience

Theme

Move from self-service platform to private cloud product.

Main promise

DADA Cloud Console becomes a full control panel for applications, managed services, access, observability, cost and automation.

Features

v4.1 Marketplace

- Application templates.
- Managed service catalog.
- One-click stacks.
- Template versioning.
- Recommended architectures.

v4.2 Object storage

- Create bucket.
- Access keys.
- S3 endpoint info.
- Policy presets.
- Usage status.

v4.3 AI/worker services

- Create worker service.
- Create AI agent service.
- Attach vector database later if needed.
- Job status.
- Queue integration.

v4.4 Advanced observability

- Built-in metrics cards.
- Logs viewer.
- Error rate.
- Latency.
- Resource usage.
- Backup health dashboard.

v4.5 Cost and usage

- Per-project usage.
- Per-app usage.
- Storage usage.
- Estimated cost.
- Plan recommendations.

v4.6 Policy-as-code integration

- Configurable platform policies.
- OPA/Kyverno/CEL integration.
- Policy previews before commit.
- Policy violation explanations.

v4.7 Disaster recovery center

- Project-level recovery view.
- Backup coverage map.
- Restore drills.
- RPO/RTO indicators.
- Cross-environment restore.

v4.8 Public API and CLI

- Public Platform API.
- CLI for developers.
- Service accounts.
- Token management.
- Terraform provider later if needed.

Exit criteria for v4

- Console feels like a real private cloud product.
- Most routine operations do not require platform engineer intervention.
- Users can provision, observe, recover and manage resources from one place.
- GitOps remains the underlying source of truth.

Part 4. Recommended first implementation slice

First vertical slice

Build one complete user journey:

`Login → Select project → Create database → Operation created → Git commit →
Argo sync → ServiceDatabase ready → UI shows status`

This is the smallest serious proof that the architecture works.

Why database first

`ServiceDatabase` already exists in production.

This reduces platform risk because the underlying CRD/model already has real usage.

Required components for first slice

Backend

- OIDC login.
- Project membership mock or simple DB table.
- Operation table.
- CreateServiceDatabase endpoint.
- Validation.
- Audit event.

Worker

- Poll queued operations.
- Render `ServiceDatabase` YAML.
- Commit to Git.
- Update operation status.

Git

- Define path convention.
- Bot credentials.
- Commit format.

Argo

- Ensure target path is watched.
- Ensure auto-sync is enabled for target environment.

UI

- Project selector.
- ServiceDatabase list.
- Create database form.
- Operation timeline.
- Status card.

First slice acceptance test

Given a user with access to project `internal`, when they create database `codex1b`, then:

1. backend creates operation;
2. worker commits `ServiceDatabase` manifest;
3. Git contains generated YAML;
4. Argo syncs the resource;

5. Kubernetes contains `ServiceDatabase` ;
 6. UI shows status as Ready or Failed;
 7. audit log links actor, operation and commit.
-

Part 5. Key product principle

DADA Cloud Console must not expose infrastructure complexity as its main interface.

The user should not think in terms of:

```
Deployment
Service
Ingress
ClusterIssuer
K10 Policy
BlueprintBinding
Secret
RoleBinding
ProviderConfig
```

The user should think in terms of:

```
Application
Database
Domain
Backup
Restore
Access
Environment
Project
```

The platform owns the complexity.

The console sells simplicity.